

The Agreement Problem



COMS4113

COMS4113: Fundamentals of Distributed Systems

Instructor: Hubertus Franke

Columbia University

<https://frankeh.github.io/cu-ds1-2026-fall>



Agreement in Distributed Systems

Two flavours, one hard core

- A set of nodes in a distributed system often needs to **agree on something**: a decision, the value of a variable, the order of events, ...
- **Examples:**
 - **Distributed mutual exclusion** (Lecture 4): nodes agree on who holds the lock at any given time.
 - **The ATM example** (Lecture 3): the ATM front-end and the banking service agree on whether to commit or abort a withdrawal.
 - **HW2**: primary and secondary replicas of a key-value store agree on the sequence of values written at each key.

Two Types of Agreement

Consensus

Participants agree on a value, and they are **willing and able to accept any** of the proposed values.

Atomic commitment

Participants agree on a value, but each has **constraints** on which values it can accept at all.

Examples from human life:

Deciding **when** to meet is likely ...

atomic commitment

Deciding **which room** to meet in is likely ...

consensus

Why: everyone has other commitments, so a proposed *time* can be genuinely impossible for someone. Any free *room* will usually do.

Which Type Is Each?

Distributed mutual exclusion: nodes agree on who holds the lock at any time.

Consensus, applied repeatedly

The ATM example: front-end and banking service agree on whether to commit or abort the withdrawal.

Atomic commitment

HW2 replication: primary and secondaries agree on the sequence of values written at each key.

Consensus, applied repeatedly

Note: a lock handoff is a *sequence* of consensus instances — one per grant — not a single one. Same for a replicated log: one instance per slot.

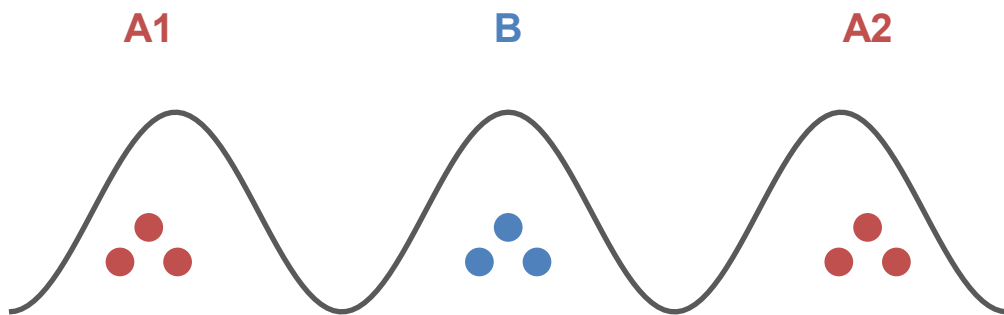
- In the **asynchronous system model**, it is impossible to guarantee agreement in finite time under all failure scenarios.
- **Consensus is tractable in practice**: protocols exist that solve it under the vast majority of plausible failure scenarios.
- **Atomic commitment is harder**. If every participant has its own constraints, a single participant's failure can leave the others unable to decide — they cannot know its vote. [Skeen-1981]
- In that sense atomic commitment is “*even harder*” than consensus, and it is why 2PC blocks (Two-Phase-Commit)

- **Focus on consensus.**
 - Justify impossibility intuitively with the standard example: the **two generals problem**.
 - Formulate the **consensus problem** abstractly.
 - Discuss the 1985 impossibility result of Fischer, Lynch and Paterson (**FLP**): consensus cannot be guaranteed in an asynchronous system if even one process may fail. [FLP-1985]
 - See **how the field escaped FLP** in practice — and what that costs.
 - Extend the failure model from **crashing** to **lying**: the Byzantine generals problem. [Lamport-1982]
 - Survey where consensus **shows up in real systems**.

The Two Generals Problem

Why agreement over an unreliable channel is hard

The Two Generals Problem



Armies **A1** and **A2** must take the fortified city **B**. Messengers between A1 and A2 must pass *through* B's territory.

Two armies, **A1** and **A2**, want to take a fortified city, **B**.

Both must attack at the same time to succeed — either army attacking alone is defeated.

The armies communicate by **messenger**, but messengers can be **captured or delayed**. Delivery is unreliable.

Three Requirements for a Solution

Consistency

Both armies decide to attack at the same time.

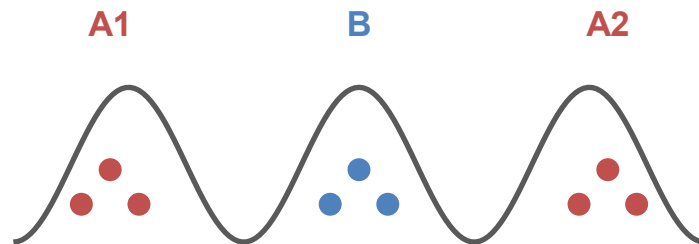
Termination

Each army decides after a **finite** number of messages.

Validity

The time to attack was **proposed by one of the armies** — not invented from nothing.

These three are the same three we will require of consensus in the abstract. Keep them in mind.



Question: what protocol achieves all three?

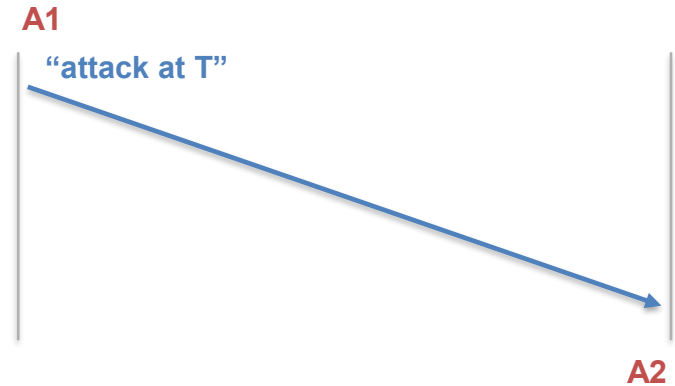
Case 1: Known Delays

Reliable delivery — the **synchronous** system model

Protocol:

- 1 Pre-agree that **A1 proposes** the attack time and **A2 accepts**.
- 2 A1 sets the time to **now + known delay + slack** for A2 to prepare.
- 3 A1 sends the time. Delivery is reliable and the delay is bounded, so A1 **knows** A2 has it.

One message. No acknowledgement needed — the delay bound is the acknowledgement.



delivery guaranteed

Case 1: Solvable

the **synchronous** system model

The problem is solvable in a synchronous network.

One message suffices. A **known delay bound** plus **reliable delivery** together let A1 compute a deadline by which A2 has certainly received the order — so no acknowledgement is needed.

Both assumptions are load-bearing. Drop either one and the argument collapses, as the next slide shows.

But real networks are not synchronous. Lecture 4's NTP material is the evidence: a bound on message delay is exactly what the internet does not offer.

Case 2: Unknown Delays

Unreliable delivery — the **asynchronous** system model

Achieving consistency, termination and validity in the asynchronous model is **provably impossible**.

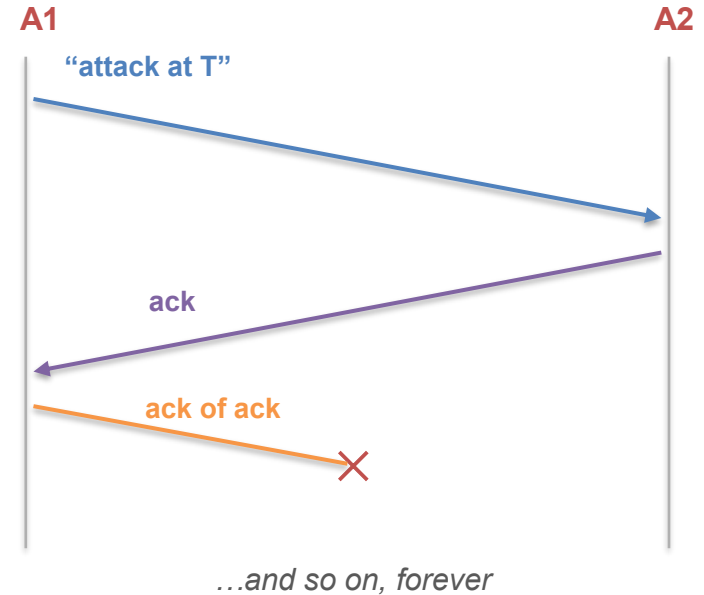
Sketch of the argument:

A1 must know A2 got the order → need an **ack**.

But the ack can be lost → A2 must know A1 got the ack → **ack the ack**.

That ack can be lost too → **ack the ack of the ack ...**

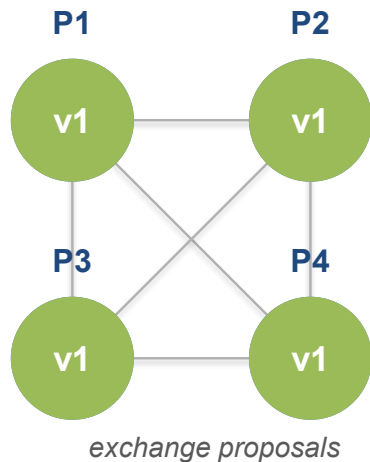
No finite protocol terminates. Neither general is ever certain.



The Consensus Problem

Stated abstractly

Consensus Problem Formulation



chosen: $V = v1$

A collection of processes P_i .

They propose values v_i (a time to attack, a client update, a lock request, ...) and send messages to exchange proposals.

Different processes may propose **different** values, and every process is willing to accept **any** of them.

Exactly **one** proposed value V is *chosen*, and eventually every process learns that one chosen value.

Consensus: Three Requirements

Consistency

Once a value is chosen (*decided*), every working process has the **same** chosen value.

Termination

Eventually a value **is** chosen.

Validity

The chosen value was **proposed by one of the processes**.

Same three as two generals

Consistency $\Leftrightarrow$ both armies attack together.
Termination $\Leftrightarrow$ decide in finitely many messages.
Validity $\Leftrightarrow$ the time came from an army.

The two generals problem *is* consensus with two processes and a lossy channel.

Safety vs liveness. Consistency and validity are *safety* properties — nothing bad ever happens. Termination is a *liveness* property — something good eventually happens. FLP, next, attacks **termination**: no protocol can guarantee it. Real protocols keep safety always and give up guaranteed liveness.

The FLP Impossibility Result

Fischer, Lynch & Paterson, 1985

The FLP Impossibility Result

In an **asynchronous** system — unordered messages, unbounded communication delay, unbounded processing delay — **no protocol can guarantee consensus in finite time** if even a **single** process may fail by stopping.

What the proof actually shows:

There is always at least one **execution** — one unlucky interleaving of message delays — in which the protocol never decides.

The adversary is the **scheduler**, not a malicious node. Nodes here only crash; they never lie.

What it does not say

- X that consensus usually fails
- X that safety must be sacrificed
- X that the problem is unsolvable *in practice*

It is a statement about **guarantees in the worst case**, not about typical behaviour.

[FLP-1985] Fischer, Lynch & Paterson. *Impossibility of Distributed Consensus with One Faulty Process*. J. ACM 32(2):374–382.

Why One Crash Is Enough

- In an asynchronous system you **cannot distinguish** a crashed process from a very slow one. There is no timeout that is provably correct, because delay is unbounded.
- So a protocol has only two options, and both fail:
 - **Wait** for the silent process → it may have crashed, and you wait forever. Termination violated.
 - **Proceed** without it → it may be alive with a different value, and decide differently. Consistency violated.
- This *indistinguishability* is the heart of FLP — and the reason **failure detection**, not agreement itself, is the real problem.



Escaping FLP

What changed between 1985 and now

Three Ways Around FLP

FLP rules out a protocol that is **always safe and always terminating** in a **fully asynchronous** model. Relax any one of those and consensus becomes solvable.

Weaken the model

Partial synchrony: delays are unbounded for a while, then bounded after some unknown time *GST* (Global Stabilization Time). Enough for a protocol that is always safe and terminates once the network behaves.
Realistic compromise: assume the network is chaotic at first, but behaves nicely eventually. [DLS-1988]

Add an oracle

Failure detectors: assume a module that eventually stops suspecting live processes. $\diamond W$ (eventually weak) suffices, and Ω is the **weakest** detector that solves consensus.

$\diamond W$: focuses on who might be alive.
 Ω : focuses purely on **leadership**.

[CT-1996] [CHT-1996]

Weaken the guarantee

Randomization: decide with probability 1 in *expected* finite time. No fixed bound on any single run, but termination is almost certain.

Trade Certain Termination with Probable Termination

[Ben-Or-1983]

In practice systems take the first two: partial synchrony plus timeout-based failure detection. All three concern *timing*, and all assume **crash** faults — under lying nodes a separate $n > 3f$ bound applies (slides 33–35).

(*) n total number of nodes ; f max number of faulty node

What Real Systems Do

Every production consensus system makes the same trade: **keep safety unconditionally, make liveness conditional.**

Safety

Always. Never two different decisions — no timing assumption needed.

Liveness

Only when the network behaves: a stable leader and a responsive majority.

Failure detection

Timeouts. Tuned, not proven — a wrong timeout costs *availability*, never *correctness*.

Leader election

Consensus's own hard core, solved by randomized timeouts to break symmetry.

Paxos Made Live [CGR-2007]: Google's report on turning the algorithm into Chubby — the algorithm was the easy part.

- **Raft (2014)** — same guarantees as Paxos, restructured for understandability: strong leader, append-only log, explicit membership change. Now the default choice in new systems (etcd, Consul, CockroachDB, TiKV). [Ongaro-2014]
- **Flexible Paxos (2016)** — the majority quorum everyone teaches is **stronger than necessary**. Only $Q1 \cap Q2 \neq \emptyset$ is required, so a larger leader-election quorum buys a smaller, faster replication quorum. [FPaxos-2016]
- **Reproducibility matters** — “Raft Refloated” (2015) re-implemented and re-measured the published results, and is a model of how to read a systems paper sceptically. [Howard-2015]
- **The theory did not change.** FLP still holds. What changed is our understanding of the *design space* it permits.

What CAP Says

Brewer's 2000 conjecture; proved by Gilbert & Lynch in 2002

C

Consistency

The service behaves like a single **atomic** read/write register: every read sees the latest write.

A

Availability

Every request to a non-failing node **eventually** receives a response.

P

Partition tolerance

The network may **drop or delay** arbitrarily many messages between groups of nodes.

The theorem: no protocol implementing an atomic read/write register can guarantee **both** consistency **and** availability in a system prone to partitions.

Careful — two senses of “consistency”. CAP's C is a property of a *replicated register*. This deck's consistency (slide 16) is a property of a *consensus decision*. Same word, different definitions.

FLP Is Not CAP

Both say “you can’t have everything”, and they are **different theorems**.

FLP (1985)		CAP (2000/02)	
Failure model	One process may crash	Failure model	The network partitions
Network	Reliable , but asynchronous	Network	Unreliable : messages dropped <i>or delayed</i>
Gives up	Termination (liveness)	Gives up	Consistency (safety) <i>or availability</i> (liveness)
Scope	A theorem about <i>finite-time guarantees</i>	Scope	A theorem about a <i>replicated register</i>

What they share: both are instances of one fact — you cannot have **safety and liveness together in an unreliable system**. That framing is Gilbert & Lynch's own.

Agreement Under Lies

The Byzantine generals problem

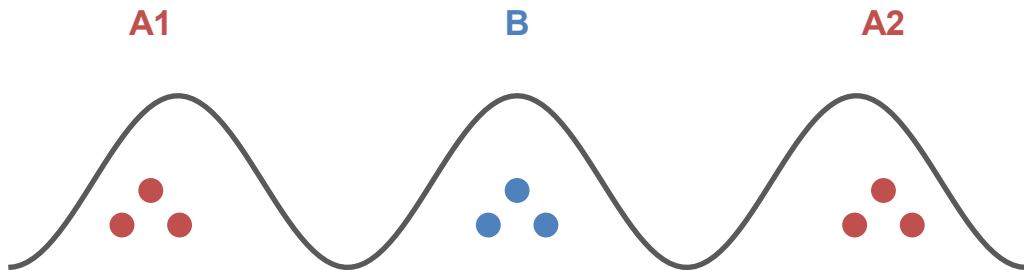
Beyond Crash Faults

Everything so far assumed a node that fails **stops**. What if it keeps running and **lies**?

Crash / fail-stop	Byzantine
A failed node sends nothing	A failed node keeps talking
Messages that arrive are truthful	Messages may be arbitrary — wrong values, wrong protocol
Every correct node sees the same message from a peer	A traitor may send different messages to different peers
Tolerate f faults with $2f+1$ nodes	Tolerate f faults with $3f+1$ nodes (proved shortly)

The key asymmetry: crash-tolerant protocols count votes. That is sound only if a node casts **one** vote and means it. A liar can vote twice, differently.

The Byzantine Generals Problem



Same terrain as before — but now **several** divisions, and some of the generals are **traitors**.

Two generals had an unreliable **channel**.
Byzantine generals have unreliable **participants**.
Next slide: why that inversion changes everything.

*“Several divisions of the Byzantine army are camped outside an enemy city, each division commanded by its own general. The generals can communicate with one another only by messenger. After observing the enemy, they must decide upon a **common plan of action**.”*

— [Lamport-1982]

Three Different “You Can't”s

We now have **three** impossibility results. They fail for **three different reasons**.

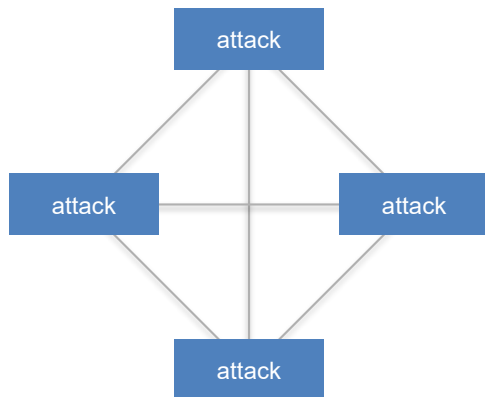
	Two generals (slide 9)	FLP (slide 18)	Byzantine, $n \leq 3f$ (slide 33)
Channel	lossy — messengers captured	reliable	reliable
Participants	honest	one may crash	f may lie
Timing	—	asynchronous	synchronous
More nodes help?	No — impossible for any n	No — one crash suffices	Yes — $n \geq 3f + 1$ solves it

The last row is the one that matters. Two generals and FLP are *impossibilities* — no amount of hardware helps. “ $n \leq 3f$ is unsolvable” is a **resilience bound**: add loyal participants and Byzantine agreement becomes solvable.

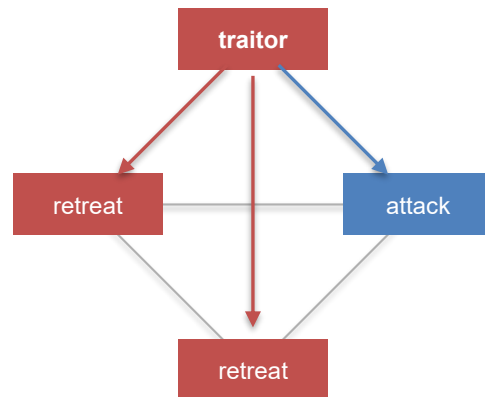
Traitors Act Arbitrarily

All loyal generals must still agree — no matter what the traitors do.

All loyal: agreement is easy



One traitor: conflicting messages

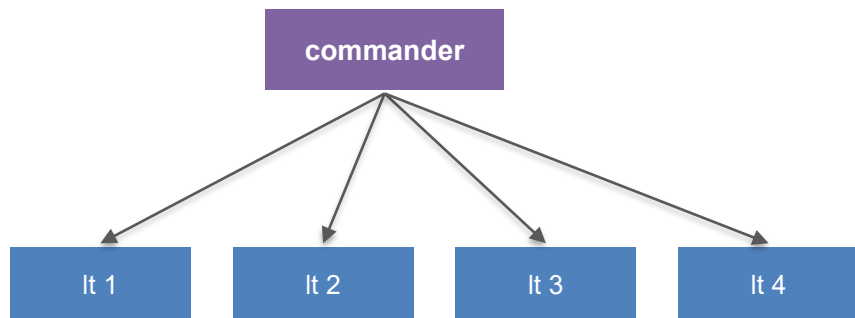


Loyal generals now disagree.

A traitor is not limited to staying silent or to sending one wrong value. It can tell **half the generals** “**attack**” and the other half “**retreat**” — and nothing in the messages themselves reveals the lie.

The Simplified Problem

Lamport reduces the symmetric problem to a **commander** sending one order to **$n-1$ lieutenants**.



one order, $n-1$ recipients

IC1

All loyal lieutenants obey the **same** order.

IC2

If the commanding general is loyal, then every loyal lieutenant obeys **the order he sends**.

*Lamport calls this pair **Interactive Consistency**.*

Why this is not a weaker problem: if the commander is a traitor, IC2 is meaningless and IC1 still binds — the loyal lieutenants must agree on *something*, even on an order nobody legitimately gave.

The reduction is lossless: solve this and you solve the symmetric problem — every general runs one instance as commander.

Same Properties, Harder Adversary

Byzantine agreement is **not a new problem**. It is our problem with the failure model changed.

Consistency	IC1 — all loyal lieutenants obey the same order	slide 16
Validity	IC2 — a loyal commander's order is the one obeyed	slide 16
Termination	Every loyal lieutenant eventually decides	slide 16

What changed: the adversary. Crash faults remove a voter; Byzantine faults give the adversary a **voice**.

Consequence for slide 21. Partial synchrony, failure detectors and randomization are escapes from *FLP*, i.e. from crash faults. None of them removes the $n > 3f$ bound proved next: it is an orthogonal constraint on *how many* nodes you need, not on timing.

Why Three Generals Fail

“If the generals can send only **oral messages**, then no solution will work unless more than $\frac{2}{3}$ of the generals are loyal.” — [Lamport-1982] (oral == sender is known + content can be altered + absence of msg can be detected)

Scenario 1: loyal commander

commander (loyal) says **retreat**

traitor lieutenant reports “commander said **attack**”

Scenario 2: traitor commander

commander (traitor) says **retreat** to one, **attack** to the other

loyal lieutenant honestly reports “commander said **attack**”

The loyal lieutenant on the left receives exactly the same two messages in both scenarios.

It must obey **retreat** in scenario 1 (IC2, loyal commander) and **attack** in scenario 2 (IC1, to match its loyal peer). It cannot do both, and it cannot tell the scenarios apart. **No protocol exists for $n = 3$, $f = 1$.**

What “Oral Messages” Assumes

The impossibility holds even under these **strong** assumptions — which is what makes it a hard result.

Every message sent is delivered correctly

The channel is **reliable** — no loss, no corruption. Contrast the two generals problem, where loss was the whole difficulty.

The receiver knows who sent it

An **authenticated channel**: a traitor cannot impersonate a loyal general.

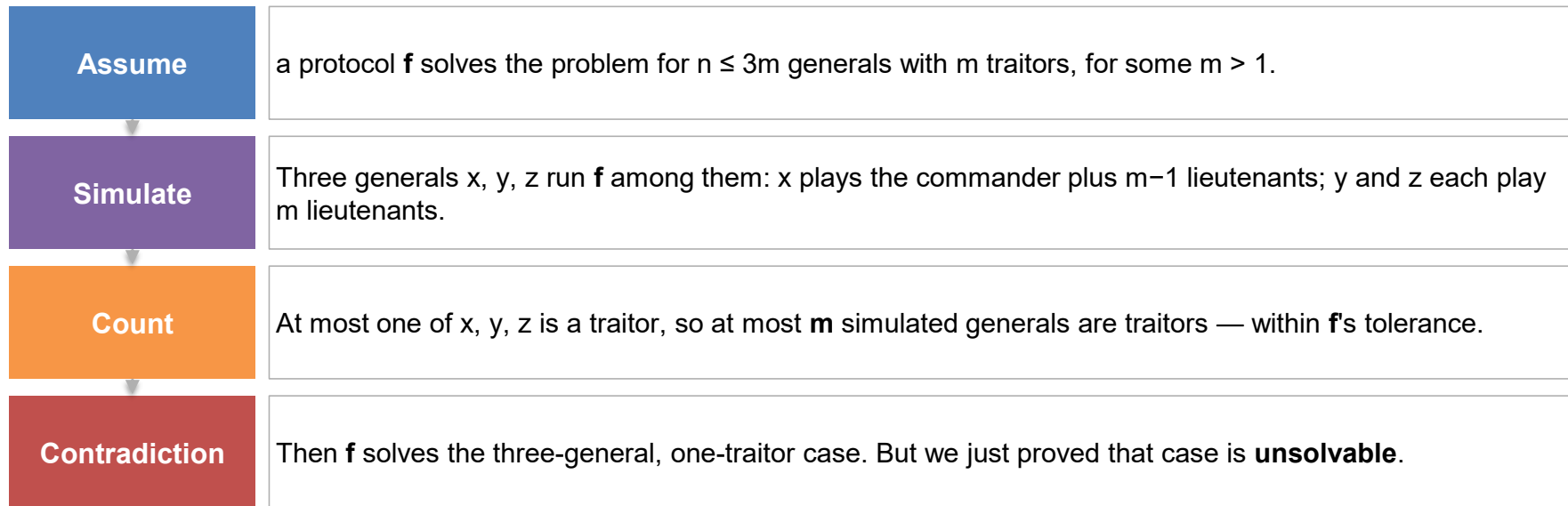
The absence of a message can be detected

A **synchronous** network: silence is detectable, so a traitor cannot stall by staying quiet.

So the difficulty is purely the lying, not delay, loss, or impersonation. Relaxing any of these three only makes it worse.

From $n = 3$ to $n = 3m$

The general bound follows from the three-general case by a **simulation argument**.



Result: with oral messages, tolerating f traitors needs $n \geq 3f + 1$.

Solution 1: Oral Messages, OM(m)

A **recursive majority** algorithm. Tolerates f traitors when $n \geq 3f + 1$.

OM(0) — base case

Commander sends its value to everyone; each lieutenant uses the value it received.

OM(k) — recursive case

1. Commander sends its value to all $n-1$ lieutenants.
2. Each lieutenant becomes commander of a smaller **OM(k-1)** among the remaining $n-1$, relaying what it received.
3. Each takes the **majority** of the $n-1$ values it collects.

Intuition

For every message received, run a smaller Byzantine agreement to tell everyone else what you heard — so a traitor's lie is outvoted by honest relays.

Cost

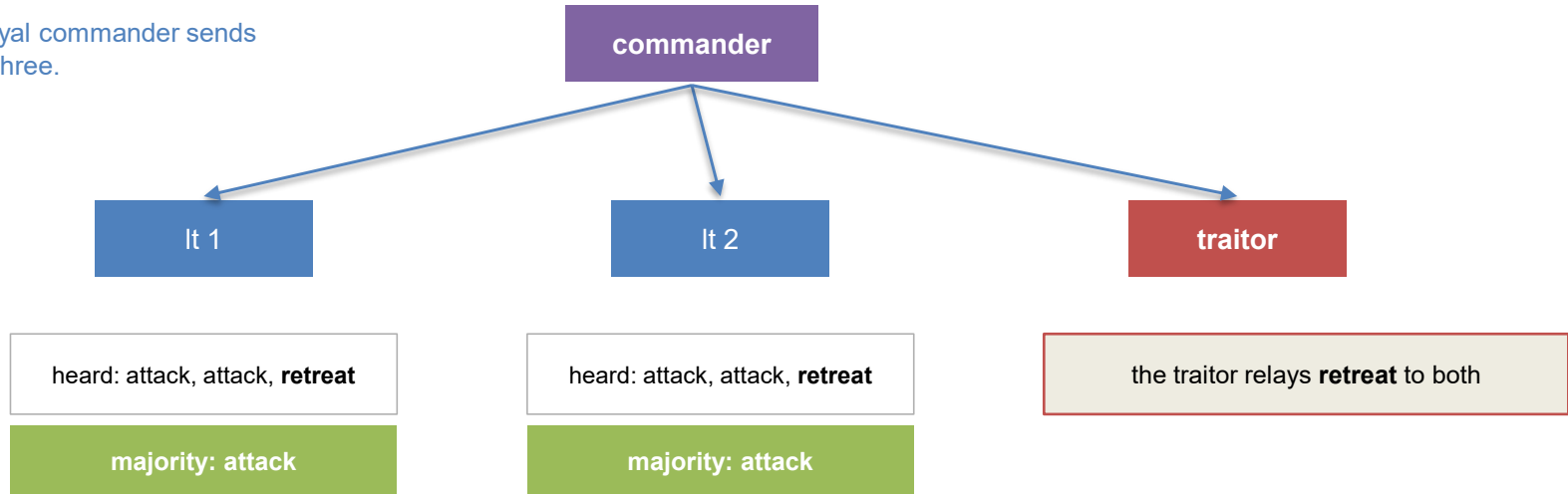
Message complexity **$O(n^m)$** — exponential in the number of faults tolerated. Correct, but impractical for even modest m .

Why a plain majority vote is not enough: each lieutenant must learn what *every other* lieutenant heard, and that second-hand report is itself subject to lying — hence the recursion.

OM(1) with Four Generals

$n = 4, f = 1$. The traitor lies to one peer; the majority still carries.

Round 1: the loyal commander sends
“**attack**” to all three.



Both loyal lieutenants collect {attack, attack, retreat} and take the majority: **attack**. IC1 holds — and since the commander was loyal, IC2 holds too. With $n = 3$ there would be no majority to take.

Solution 2: Signed Messages, SM(m)

Give the generals **unforgeable signatures** and the $\frac{1}{3}$ bound disappears.

What signatures add

- A signed order **cannot be forged**
- Anyone can **verify** who signed it

What that removes

A traitor can no longer **misreport** what it received — the signature is the proof.

SM(m) — sketch

Each lieutenant keeps a set **V(i)** of orders it has seen. The commander signs its order and sends it. On receiving a new order carrying a chain of k signatures, a lieutenant adds it to $V(i)$ and — if $k < m$ — appends its own signature and forwards it to lieutenants not already in the chain. When no more messages arrive, decide **choice(V(i))**: the single value if $|V(i)| = 1$, otherwise **retreat**.

Resilience: works for **any** number of traitors — $n \geq f + 2$ suffices.

Cost: $O(n^2)$ messages — far better than OM(m)'s $O(n^m)$.

How Many Nodes Do You Need?

Minimum n for which an f -resilient consensus protocol exists.

	Synchronous	Asynchronous	Partially synchronous
Fail-stop	$f + 1$	∞	$2f + 1$
Crash	$f + 1$	∞	$2f + 1$ (Paxos)
Byzantine, signed messages	$f + 1$ (SM)	∞	$3f + 1$ (PBFT)
Byzantine, auth. channel	$3f + 1$ (OM)	∞	$3f + 1$

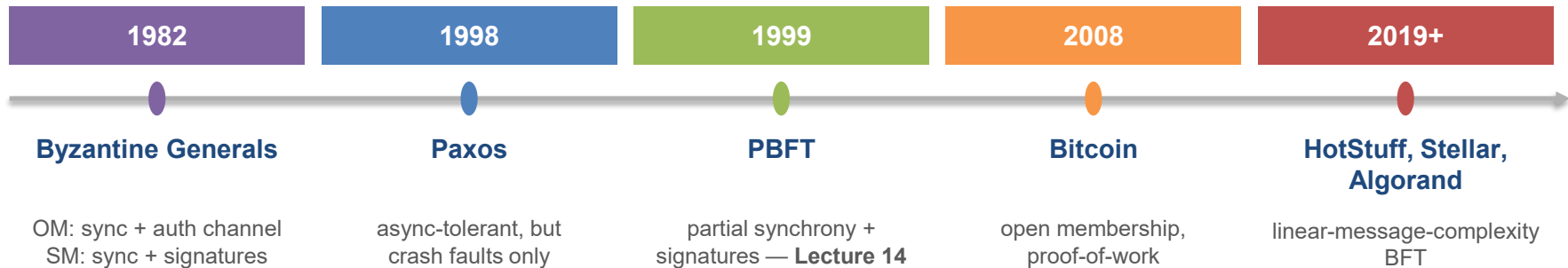
this column is FLP

Partial synchrony: bounds exist but are not known a priori.

PBFT : Practical Byzantine Fault Tolerance

Byzantine Agreement: The Arc

Where this leads — and what the rest of the course does with it.



Closed vs open membership. PBFT needs a **fixed, known** set of replicas and counts fixed-size quorums. Blockchains drop that assumption and replace quorums with **proof-of-X**.

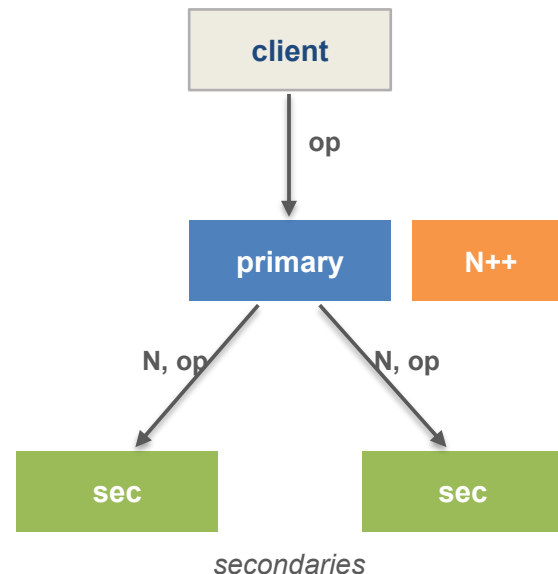
Next on this thread: Lecture 14 builds PBFT's pre-prepare / prepare / commit / reply protocol on the $3f + 1$ bound you just derived.

Consensus Is Pervasive

Where this shows up in real systems

Replicated State: Ordering Updates

- **Agreeing on the order of updates to a replicated database.**
 - One solution is **primary/secondary replication** (as in HW2).
 - Several replicas; one is the **primary**.
 - Reads and writes go to the primary, which assigns each operation a **sequence number** before forwarding to the secondaries.
 - Many variants exist, but they all reduce to one core consensus question: **how do we choose the primary?**
 - a.k.a. **leader election**.



More Consensus in Disguise



COMS4113

- **Grabbing a lock for mutual exclusion.** Lecture 4 gave a specific algorithm; at its core it is consensus, and a generic consensus protocol solves it.
- **Reliable, ordered multicast.** All members of a group agree on the set *and* the order of messages they deliver.
- **Group membership.** Agreeing on who is currently in the cluster is itself a consensus problem — and the one that bites hardest, since the membership list is what quorums are counted over.
- **Distributed locking services** — Chubby, ZooKeeper, etcd — are consensus engines exposed as a lock/config API.
- ... and many others. **One protocol that solves consensus solves them all.**

When Is Consensus Guaranteed?

- Four **determining factors** — the axes of the table on the next slide:
 - **Processors**: synchronous vs. asynchronous — is there a bound on relative processing speed?
 - **Communication**: bounded vs. unbounded — is there a bound on message delay?
 - **Messages**: ordered vs. unordered — are messages delivered in the order sent?
 - **Transmission**: broadcast vs. point-to-point — does one send reach everyone atomically?
- Sixteen combinations. Only some admit a consensus protocol.

Can's and Cannot's

Processors	Unordered		Ordered		Communication	
	Point-to-point	Broadcast	Broadcast	Point-to-point		
Asynchronous	No	No	Yes	No	Unbounded	asynchronous system (realistic): mostly no
	No	No	Yes	No	Bounded	
Synchronous	Yes	Yes	Yes	Yes	Bounded	synchronous system (not realistic): yes
	No	No	Yes	Yes	Unbounded	

Transmission

The one column that rescues asynchrony: ordered broadcast.

Table redrawn from [Turek-Shasha-1992], Fig. 1.

Three things the grid tells us that are worth stating explicitly:

Asynchrony is the killer, not delay bounds.

Bounding *communication* does not rescue the asynchronous rows. Bounding *processing* (row 3) does. This is FLP's shape, in a table.

Ordered broadcast is the one exception — and it is not free.

Atomic ordered broadcast is **equivalent** to consensus: each can implement the other. Assuming it is assuming your answer.

Only the fully synchronous, bounded row is unconditionally yes.

And no real network offers it. This is why the rest of the course studies protocols that are safe always and live usually.

Next: what to do about it.

- **Agreement** is a family of problems, not one problem. It splits into **consensus** (any value acceptable) and **atomic commitment** (participants hold vetoes).
- Both demand **consistency, termination, validity**. The two generals problem is consensus with two nodes and a lossy channel.
- **FLP**: in an asynchronous system with one possible crash, no protocol guarantees consensus in finite time. The obstacle is that **crashed and slow are indistinguishable**.
- The escape is to **change the model** — partial synchrony, failure detectors, or randomization — never to weaken safety.
- Real systems are therefore **always safe, conditionally live**. A mistuned timeout costs availability, not correctness.
- **Byzantine** faults keep the same three properties but give the adversary a voice: $n \geq 3f + 1$ with oral messages, or $f + 1$ with signatures — both only under synchrony.
- Consensus is **pervasive**: replication order, locking, membership, multicast. Solve it once and you solve them all.

- **Today:** a parenthesis on **local transactions**.
- **Then: consensus** protocols — Paxos, and Raft as its understandable restatement.
- **Then: atomic commitment** protocols — 2PC, and why it blocks.
- **Then: case studies** from industry — Spanner and friends.
- **Later: Byzantine** state-machine replication — PBFT, built on the $3f + 1$ bound derived today.

- **[FLP-1985]** Fischer, Lynch & Paterson. *Impossibility of Distributed Consensus with One Faulty Process*. J. ACM **32(2):374–382**, 1985.
- **[Turek-Shasha-1992]** Turek & Shasha. *The Many Faces of Consensus in Distributed Systems*. IEEE Computer **25(6):8–17**, June 1992.
- **[DLS-1988]** Dwork, Lynch & Stockmeyer. *Consensus in the Presence of Partial Synchrony*. J. ACM **35(2):288–323**, 1988.
- **[CT-1996]** Chandra & Toueg. *Unreliable Failure Detectors for Reliable Distributed Systems*. J. ACM **43(2):225–267**, 1996.
- **[CHT-1996]** Chandra, Hadzilacos & Toueg. *The Weakest Failure Detector for Solving Consensus*. J. ACM **43(4):685–722**, 1996.
- **[Ben-Or-1983]** Ben-Or. *Another Advantage of Free Choice*. PODC 1983, pp. 27–30.
- **[Skeen-1981]** Skeen. *Nonblocking Commit Protocols*. SIGMOD 1981, pp. 133–142.
- **[Lamport-1982]** Lamport, Shostak & Pease. *The Byzantine Generals Problem*. ACM TOPLAS **4(3):382–401**, July 1982.
- **[PSL-1980]** Pease, Shostak & Lamport. *Reaching Agreement in the Presence of Faults*. J. ACM **27(2):228–234**, 1980.

Further Reading: In Practice



COMS4113

- **[Ongaro-2014]** Ongaro & Ousterhout. *In Search of an Understandable Consensus Algorithm* (Raft). USENIX ATC 2014, pp. **305–319**.
- **[FPaxos-2016]** Howard, Malkhi & Spiegelman. *Flexible Paxos: Quorum Intersection Revisited*. OPODIS 2016, **25:1–25:14**.
- **[CGR-2007]** Chandra, Griesemer & Redstone. *Paxos Made Live — An Engineering Perspective*. PODC 2007, pp. **398–407**.
- **[Howard-2015]** Howard, Schwarzkopf, Madhavapeddy & Crowcroft. *Raft Refloated: Do We Have Consensus?* SIGOPS OSR **49(1):12–21**, 2015.
- **[Gilbert-Lynch-2002]** Gilbert & Lynch. *Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services*. SIGACT News **33(2):51–59**, 2002.
- **[Gilbert-Lynch-2012]** Gilbert & Lynch. *Perspectives on the CAP Theorem*. IEEE Computer **45(2):30–36**, 2012 — the authors' own retrospective; source of the CAP definitions used in this deck.
- **[Castro-1999]** Castro & Liskov. *Practical Byzantine Fault Tolerance*. OSDI 1999 — journal version *Practical Byzantine Fault Tolerance and Proactive Recovery*, ACM TOCS **20(4):398–461**, 2002. Covered in **Lecture 14**.
- **[Herlihy-1991]** Herlihy. *Wait-Free Synchronization*. ACM TOPLAS **13(1):124–149**, 1991 — consensus numbers, the shared-memory counterpart to FLP.